

Chitty Kampany — Product & Architecture Brief

Prepared 20 August 2026

What this is

Chitty Kampany is a small, open-source, low-tech tool for digitizing how a chit fund committee runs its daily collections and draws. It replaces the paper registers a field agent currently carries with a phone-based collection screen, and gives the committee a live, auditable ledger instead of reconciling notebooks by hand. The name is a nod to the “kampany” (company) that runs the chits.

The problem it solves

A chit fund committee runs several chits at once, each with its own members, installment amount, and schedule. Collection agents go door to door daily, taking cash or UPI payments, and at some interval a draw decides who receives that round’s pot. Today this lives on paper: easy to lose, slow to reconcile, and hard for a member to trust that their payment history is accurate. The goal is not a feature-rich fintech product — it is digital, easy, and transparent, in that order, and it has to run without an IT team maintaining servers.

Who uses it

Three roles. The **collection agent** works from a phone in the field, picking a chit, picking a member, and logging what was collected, in real time, for however many chits they cover that day. The **committee/admin** sets up each chit’s rules, enrolls members before it starts, records the outcome of each physical draw, and watches who is current or in default. A **member** is, for now, someone the committee tracks on their behalf — read-only self-service for members is a deliberate v2 item, not v1.

How a chit works here

Every chit is its own configuration, not a hardcoded formula. A chit has a fixed installment amount, a collection frequency (daily, working-days-only, alternate days, weekly, monthly, or a custom calendar), a fixed number of participants who all join before the chit starts, and a draw interval expressed in collection cycles. Each round’s pool is the sum of that round’s installments across all active members; the committee conducts the physical draw itself and records the winner in the app, chosen only from members who have not already won in that chit. The commission the committee earns is also configurable per chit — as an extra installment collected once per member over the life of the scheme (the model this brief was designed against), a flat fee, or a percentage of the pool — rather than assumed fixed at any particular number. A configurable holiday calendar sits underneath the working-day logic, so festival and bank holidays can be excluded from “working day” schedules on top of Sundays.

V1 feature set

The agent’s screen lets them sign in with their own Google account, choose a chit, choose a member from that chit’s active roster, and log a payment with its amount and mode (cash or UPI), with a safeguard against two agents’ submissions colliding at the same instant. The admin panel lets the committee create a chit with all its parameters, enroll members into it before it starts, record each draw’s winner and see the computed pool and payout beforehand, and review a ledger per chit, per member, or per agent — including a straightforward view of who has not paid what they owe as of today. Member onboarding, whether done by the admin or an agent, captures phone number and email address up front, since both feed the notification flow described below.

Starting a new chit is a short guided flow rather than a single form: the admin first defines the chit’s rules (installment amount, frequency, draw interval, participant count, commission rule, start date), then adds members up to the configured capacity — each new member can also be enrolled in other chits from the same screen — and finally reviews and activates the chit, which locks further enrollment the moment it starts, matching the “no joining after it begins” rule.

A small dashboard gives the committee a live read on collection health: today’s total collected across all chits, each chit’s collected-to-date against what’s expected by now, a defaulter count, and a per-agent breakdown of what each agent brought in today. It’s built with Google’s native charting (Google Charts inside the Apps Script HTML page), so it needs no external charting service or added dependency.

Every one of these functions is gated by who’s signed in. A **Users** list holds each person’s Google account email against a role — admin or agent — and every admin function (creating a chit, enrolling members, recording a draw, viewing the dashboard) checks that role before running; an agent’s account only ever sees the collection screen for members on chits they’re assigned to. Someone not on the list at all gets turned away outright.

Payment and draw notifications go out through two channels with very different effort levels. Email is fully automatic in v1 — Apps Script’s own mail service sends a receipt on each logged payment and a result notice after each draw, at no cost and no external setup. WhatsApp starts as a manual step: the app builds a pre-filled WhatsApp message (via a [wa.me](#) link) with the same receipt or draw result, and the admin or agent taps it to actually send — no Business API account, verification, or per-message cost required for v1. Full automatic WhatsApp sending is feasible later through the WhatsApp Business Cloud API, but it needs Meta business verification, a dedicated WhatsApp number, pre-approved message templates, and an ongoing per-message cost, so it’s treated as a deliberate v2

upgrade rather than a v1 default.

Parked for v2

A read-only member lookup so someone can check their own payment history without going through the committee. Fully automatic WhatsApp sending via the Business API. Exportable reports. Support for more than one committee sharing the same deployment. None of these are ruled out — they are simply not needed to prove the core loop works.

Architecture

The system runs on Google Sheets as the data store and Google Apps Script as the application layer, rather than a self-hosted web app on a server. This is a deliberate choice, not a shortcut: there is no IT team to keep a server patched and backed up, Google Sheets is free at this scale and already something a committee member can open and inspect directly, and it keeps the whole system inspectable in a way that a database hidden behind an app is not. The open-source distribution model fits naturally too — the script source lives in a public GitHub repository, managed with `clasp` for real version control, and anyone running their own chit fund copies the template spreadsheet, pastes or pushes the script in, and deploys their own private web app. No shared infrastructure, no hosting bill, no single point of failure for every group using it.

The spreadsheet holds eight tabs functioning as tables. **Chits** carries each chit’s configuration — name, installment amount, frequency type, draw interval, participant count, commission rule, and status. **Members** holds member identity, phone, and email. **Enrollments** links members to the chits they joined, timestamped before each chit’s start. **Collections** is the append-only payment log — chit, member, date, amount, mode, and the agent who recorded it. **Draws** records each round’s winner, the computed pool, the commission taken, and the net payout. **Holidays** holds the configurable non-collection dates. **Users** maps each person’s Google account email to a role, admin or agent, and is the single source of truth the access-control layer checks against.

Four script modules sit on top: a working-day/frequency engine that turns a chit’s schedule configuration into actual due dates, a chit engine that computes round pools, draw eligibility, and commission from that same configuration, a notification module that sends email directly through Apps Script’s mail service and builds the tap-to-send WhatsApp links, and the two Apps Script web app front ends — one HTML interface for agents, one for the committee, each rendering only the functions its signed-in user’s role permits — both reading and writing through Apps Script’s `LockService` so concurrent submissions from multiple agents can’t corrupt a row. Agent identity comes from their signed-in Google account rather than a typed name, so every collection is attributable without extra data entry.